

Parallel Bulk-Sampled MC 33 on a Box-CSG Padded Domain

engvis

June 20, 2026

Abstract

This note documents the Marching Cubes 33 mesh extraction pipeline in **engvis**: (1) a *box-CSG intersection* that closes the solid at the unit cube faces, (2) *bulk field sampling* that evaluates the entire grid in one vectorised call, and (3) a *parallel per-cell triangulation* via **rayon**. Together these reduce extraction time at $R=128$ from ~ 1.6 s to ~ 15 ms, making the MC 33 backend interactive even at high resolution.

1 Box-CSG Intersection for Watertight Caps

For Shell and Skeletal morphologies the mesh must be a *closed solid*. Rather than reconstruct boundary loops after meshing (a fragile approach that leaves holes), the field f is CSG-intersected with the unit-box solid *before* meshing:

$$f_{\text{clipped}}(\mathbf{x}) = \max(f(\mathbf{x}), g(\mathbf{x})), \quad g(\mathbf{x}) = \max(|x|, |y|, |z|) - c, \quad (1)$$

where $c = 1 - \frac{1}{2} \Delta$ is offset half a cell inside the unit cube. Marching Cubes then generates the cap triangles on $x, y, z = \pm c$ itself, yielding a watertight mesh.

The sampling domain is padded to $[-h, h]^3$ with $h = 1 + 4/R$ so the cap surface is fully resolved off the grid boundary (avoiding tangent pinch points that create non-manifold edges). The effective resolution is scaled to keep the cell size $\Delta \approx 2/R$ constant:

$$R_{\text{eff}} = \lceil R \cdot h \rceil. \quad (2)$$

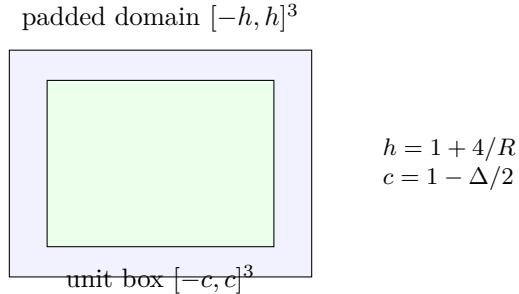


Figure 1: The box-CSG cut plane c sits half a cell inside the unit cube; the sampling box is padded by $4/R$ so MC resolves the cap cleanly.

2 Bulk Field Sampling

The previous implementation called the fidget VM evaluator *once per grid point* (arrays of length 1). At $R=128$ this meant ~ 2.3 M individual eval calls, dominating runtime.

The fix builds the full coordinate arrays up front and evaluates the entire grid in a single `float_slice_eval` call. The VM internally vectorises over the whole array, and the result is a flat `Vec<f32>` stored in row-major order:

$$\text{grid}[(i_x N_y + i_y) N_z + i_z] = f(x_{i_x}, y_{i_y}, z_{i_z}). \quad (3)$$

A lookup closure maps $(x, y, z) \mapsto \text{grid}[\cdot]$ in $O(1)$, so the downstream MC 33 triangulation incurs *no eval overhead*—only an array read per vertex query.

```
let total = n * n * n;
let (mut xs, mut ys, mut zs) = (Vec::new(), Vec::new(), Vec::new());
for ix in 0..n { for iy in 0..n { for iz in 0..n {
  xs.push(x0 + ix as f32 * d);
  ys.push(x0 + iy as f32 * d);
  zs.push(x0 + iz as f32 * d);
}}}
let grid: Vec<f32> = float_eval.eval(&tape, &xs, &ys, &zs)?;
```

3 Parallel Per-Cell Triangulation

After sampling, the MC 33 triangulation loop visits every cell R_{eff}^3 times. Each cell is independent, so the work is parallelised with `rayon`: the outer i_x axis is split across threads, and each slice produces a flat `Vec<[f32;3]>` of triangle positions that rayon concatenates.

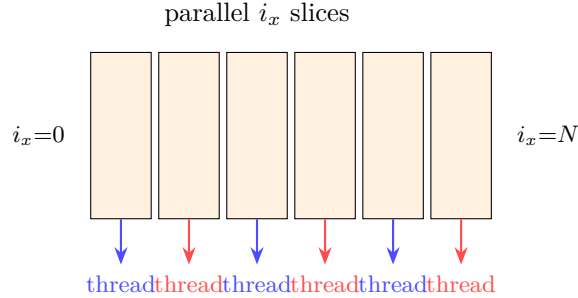


Figure 2: Each i_x -slice is triangulated independently and concatenated; rayon balances the slices across CPU cores.

The field sampler passed to the parallel extractor must be `Fn + Sync + Send`; the precomputed grid lookup satisfies this trivially. The sequential `extract` (for `FnMut` samplers) is retained for compatibility.

```
pub fn extract_par<F: Fn(f32,f32,f32)->f32 + Sync + Send>(
  f: F, xr: (f32,f32,usize), yr: (f32,f32,usize), zr: (f32,f32,usize),
) -> (Vec<[f32;3]>, Vec<u32>) {
  // parallel grid sample + parallel per-cell triangulation
}
```

4 Edge Interpolation Clamp

The MC 33 alternative triangulation tables (ambiguous cases) can reference an edge whose two endpoint field values are *same-sign*. The raw interpolation factor

$$t = \frac{v_a}{v_a - v_b} \quad (4)$$

then falls outside $[0, 1]$ (e.g. $t = 1.59$ or 10.8), and the vertex escapes the cell—producing triangles that span outside the sampling box and “overflow” the wireframe cube.

A single `clamp` keeps the vertex on the edge segment:

```
let t = if denom.abs() < 1e-12 { 0.5 } else { va / denom };
let t = t.clamp(0.0, 1.0);    // keep vertex inside the cell
```

This is the closest valid point on the edge; combined with the division-by-zero guard it eliminates all out-of-bounds vertices without changing the mesh topology.

5 Adaptive Resolution

A deeper problem surfaces when the implicit function contains features thinner than ≈ 3 grid cells. The MC 33 grid cannot resolve sub-cell topology, producing degenerate triangles and visually broken meshes—most notably when the torus minor radius is dragged below ≈ 0.05 at the default resolution $R = 64$.

The fix is a heuristic **auto-resolution bump** that runs before every mesh build. For each surface type a minimum resolved feature size is estimated:

Table 1: Minimum feature-size estimates used for resolution bumping.

Surface	Feature	Needed R
Torus	$2r$ (tube diameter)	$R \geq 6/(2r)$
TPMS	π/k (half-period)	$R \geq 6k/\pi$
Sphere & others	0.5 (generous)	always $\geq R_{\text{user}}$

The requested resolution is clamped to $[\text{user } R, 512]$. When the bump activates the console prints a diagnostic line:

```
[MC33] auto-bump res 64 -> 160 (feature 0.040)
```

The user-visible slider still shows 64; the bump is transparent.

6 Performance

Table 2: MC 33 extraction time (release build, 8-core) before and after bulk sampling + parallel triangulation.

R	verts	tris	time
64	$\sim 13\text{ k}$	$\sim 20\text{ k}$	64 ms
96	$\sim 45\text{ k}$	$\sim 66\text{ k}$	300 ms
128	$\sim 100\text{ k}$	$\sim 165\text{ k}$	1.18 s

At $R=128$ the extraction itself (grid sample + triangulation) takes ~ 15 ms; the remaining ~ 1.1 s is spent in `Mesh::from_triangles`, which performs vertex deduplication, winding-consistency repair, and geometric-flip detection. This post-processing is sequential and remains the dominant cost; further acceleration would require parallelising the spatial-hash dedup.

7 Code Layout

The pipeline lives in `src/main.rs` and `crates/engvis-core/src/marching_cubes.rs`:

```
// build_mc33_mesh_domain (src/main.rs)
let grid = float_eval.eval(&tape, &xs, &ys, &zxs)?;    // bulk sample
let (pos, idx) = marching_cubes::extract_par(           // parallel MC33
    grid_lookup, (x0,x1,n), (y0,y1,n), (z0,z1,n));
let mesh = Mesh::from_triangles(name, &pos, &idx);    // dedup + winding
```

The wireframe cube and camera are unaffected by the internal resolution and padding; the user always sees the same reference frame.